



HCMUS

Viet Nam National University
Ho Chi Minh City
University of Science

CSC10006 – Introduction to Database

Basic query

(Practicing SQL through simple query exercises)

Bộ môn HTTT - Khoa CNTT - ĐHQG-HCM

TABLE OF CONTENTS

1	Objectives.....	1
2	Detailed Instructions.....	1
2.1.1	General SQL Query Syntax.....	1
2.1.2	Simple Queries.....	2
2.1.3	Queries with Sorting.....	4
2.1.4	Queries with Simple Conditions.....	5
3.1.1	Queries with String Conditions.....	8
3.1.2	Queries with Date and Time Conditions.....	9
3.1.3	Using Functions in Queries.....	10
3.1.4	Supported Set Operators in SQL Server.....	11
3.1.5	Join Operations.....	12
3.1.6	Using ALIAS.....	13
3.1.7	Using JOIN.....	15
4	In-Class Exercises.....	15

1 Objectives

- In this lab, students will use SQL to answer simple data query questions.
- After completing this lab, students will be able to:
 - o Understand the basic syntax of SQL queries
 - o Write and execute simple SQL queries

2 Detailed Instructions

2.1.1 General SQL Query Syntax

In general, an SQL query consists of three main clauses:

- **SELECT**: Specifies the columns to be displayed in the query result.
- **FROM**: Specifies the tables from which data is retrieved.
- **WHERE**: Specifies the conditions used to filter records.

To extend the functionality of SQL queries, the SELECT–FROM–WHERE block can be further combined with additional clauses such as **GROUP BY**, **HAVING**, and **ORDER BY**, as well as aggregate functions including **MAX**, **MIN**, **COUNT**, **SUM**, and **AVG**, ...

The general syntax of a data query is as follows:

```
SELECT [modifier] <attribute_list_1>  
FROM <table_list or query/view [AS alias]>  
[WHERE <condition_1>]  
[GROUP BY <attribute_list_2>]  
[HAVING <condition_2>]  
[ORDER BY <attribute_list_3> [ASC | DESC]]
```

Explanation:

- **SELECT**: Specifies the attributes to be returned in the result set, optionally with modifiers (e.g., **DISTINCT**).
- **FROM**: Specifies the tables, queries, or views from which data is retrieved, with optional aliases.

- **WHERE:** Filters records based on specified conditions.
- **GROUP BY:** Groups records based on one or more attributes.
- **HAVING:** Applies conditions to grouped records.
- **ORDER BY:** Sorts the result set in ascending (**ASC**) or descending (**DESC**) order
 - By default, sorting is performed in ascending order. When multiple columns are specified, sorting is applied from left to right in priority order.
- **Modifier**
 - **ALL:** Selects all rows from the table.
 - **DISTINCT:** Removes duplicate rows from the result set.
 - **TOP <n>:** Selects the first n rows that satisfy the query conditions.
- **Attribute List 1:** Specifies the attributes (columns) to be retrieved.

Notes:

- Attributes are separated by commas (,).
- To select all attributes of a table tbl, use tbl.*.
- If there is only one table in the FROM clause and all its columns are required, use SELECT *.
- If an attribute appears in more than one table listed in the FROM clause, the table name (or alias) must be specified to avoid ambiguity.
- **Alias**
 - An alias is a short or alternative name for a table.
 - It is useful when table names are long or required when the same table is used more than once in a query.

2.1.2 Simple Queries

SELECT <attribute_list>

FROM table_name

- The clause **SELECT *** is used to retrieve all columns of a table.
- The keyword **AS** is used to assign an alias (rename) to a column.

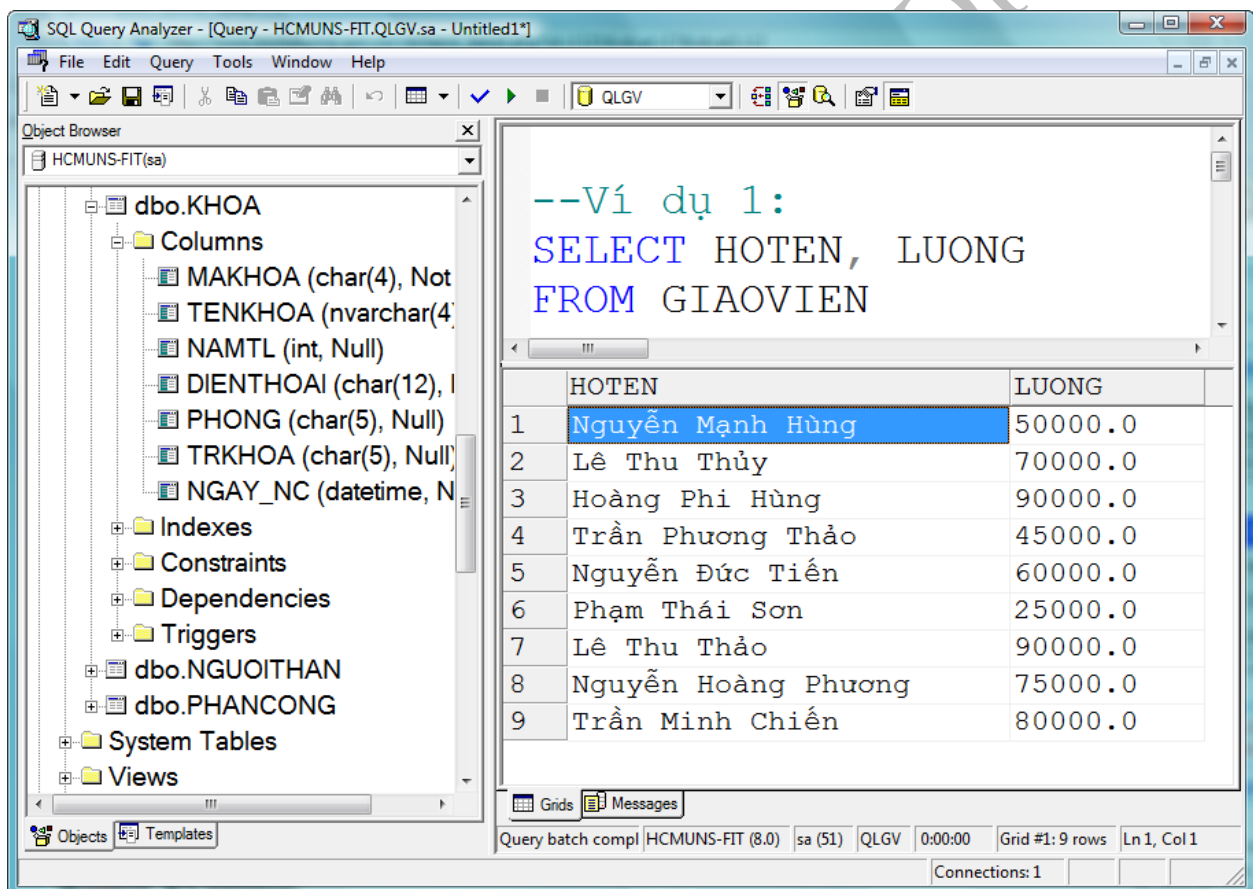
- The keyword **DISTINCT** is used to eliminate duplicate rows, while **ALL** is used to retrieve all rows. If no keyword is specified, **ALL** is applied by default.
- Arithmetic expressions such as +, -, *, and / can be used after SELECT, and these operators can be applied directly to attributes.

Example 1: Retrieve the full name and salary of all teachers.

```
SELECT HOTEN, LUONG
```

```
FROM GIAOVIEN
```

- This query returns the full name and salary of all teachers stored in the GiaoVien table:



The screenshot shows the SQL Query Analyzer interface. The Object Browser on the left displays the database structure, including the 'dbo.KHOA' table with columns like MAKHOA, TENKHOA, NAMTL, DIENTHOAI, PHONG, TRKHOA, and NGAY_NC. The main window displays the following query:

```
--Ví dụ 1:
SELECT HOTEN, LUONG
FROM GIAOVIEN
```

The results are shown in a grid with 9 rows and 2 columns: HOTEN and LUONG. The first row is highlighted in blue.

	HOTEN	LUONG
1	Nguyễn Mạnh Hùng	50000.0
2	Lê Thu Thủy	70000.0
3	Hoàng Phi Hùng	90000.0
4	Trần Phương Thảo	45000.0
5	Nguyễn Đức Tiến	60000.0
6	Phạm Thái Sơn	25000.0
7	Lê Thu Thảo	90000.0
8	Nguyễn Hoàng Phương	75000.0
9	Trần Minh Chiến	80000.0

The status bar at the bottom indicates: Query batch compl | HCMUNS-FIT (8.0) | sa (51) | QLGV | 0:00:00 | Grid #1: 9 rows | Ln1, Col1 | Connections: 1

Example 2: Retrieve the list of all teachers in the school.

```
SELECT *
```

```
FROM GIAOVIEN
```

Example 3: Retrieve the full name and salary of all teachers and rename the output columns.

```
SELECT HOTEN AS HOTEN_GV, LUONG AS LUONGGV
FROM GIAOVIEN
```

Explanation: AS is used to assign new names to the output columns.

2.1.3 Queries with Sorting

In SQL, searching can also involve sorting the result set using the **ORDER BY** clause

```
SELECT...
FROM...
ORDER BY attribute_1 [ASC | DESC], attribute_2 [ASC | DESC], ...
```

Sorting priority: Sorting is applied from left to right. That is, the result set is first sorted by the leftmost attribute. If there are ties, sorting continues with the next attribute on the right.

Example 4: Retrieve the list of teachers (full name, gender, salary) sorted in descending order of salary

```
SELECT HOTEN, PHAI, LUONG
FROM GIAOVIEN
ORDER BY LUONG DESC
```

Example 5: Retrieve the full name, department code, and salary of teachers. Sort the result in ascending order by department code and descending order by salary.

```
SELECT HOTEN, MABM, LUONG
FROM GIAOVIEN
ORDER BY MABM, LUONG DESC
```

HOTEN	MABM	LUONG
Hoàng	HTTT	80000
Dũng	KHMT	60000
An	KHMT	70000



HOTEN	MABM	LUONG
An	KHMT	70000
Dũng	KHMT	60000
Thủy	HTTT	90000

Thủy	HTTT	90000
------	------	-------

Hoàng	HTTT	80000
-------	------	-------

2.1.4 Queries with Simple Conditions

To perform conditional searches, SQL uses the WHERE clause within the SELECT statement, as shown in the following syntax:

```
SELECT ...
FROM ...
WHERE (Condition 1) AND/OR (Condition 2) .... (Condition n)
```

3 Comparison Operators

SQL supports the following comparison operators:

- > (greater than)
- < (less than)
- = (equal to)
- !=, <> (not equal to)
- >= (greater than or equal to)
- <= (less than or equal to)

Example 6: Retrieve teachers whose salary is greater than 50,000.

```
SELECT *
FROM GIAOVIEN
WHERE luong > 50000
```

Example 7: Retrieve teachers whose gender is male

```
SELECT *
FROM GIAOVIEN
WHERE PHAI = 'Nam'
```

a. AND, OR and NOT

SQL supports the following logical operators:

- **AND**: all conditions must be satisfied

- **OR**: at least one condition must be satisfied
- **NOT**: negates a condition

Example 8: Retrieve teachers from the Information Systems department (HTTT) whose salary is greater than 40,000.

```
SELECT *
FROM GIAOVIEN
WHERE MABM= 'HTTT' AND LUONG > 40000
```

Example 9: Retrieve teachers who are not in the Information Systems department (HTTT) and have a salary of 40,000 or less.

```
SELECT *
FROM NHANVIEN
WHERE NOT (MABM = 'HTTT') AND NOT (LUONG > 40000)

OR:

SELECT *
FROM NHANVIEN
WHERE (NOT (MABM = 'HTTT')) AND (NOT (LUONG > 40000))

OR:

SELECT
FROM NHANVIEN
WHERE (MABM != 'HTTT') AND (LUONG <= 40000)
```

b. BETWEEN...AND , NOT BETWEEN ... AND

Example 10: Retrieve teachers who were born between 1955 and 1960

```
SELECT *
FROM GIAOVIEN
WHERE NG SINH BETWEEN '1/1/1955' AND '12/31/1960'

or:

SELECT *
FROM GIAOVIEN
```



```
WHERE year(NGSINH) BETWEEN 1955 AND 1960
```

Or:

```
SELECT *
```

```
FROM GIAOVIEN
```

```
WHERE year(NGSINH) >= 1955 AND year(NGSINH) <=1960
```

Explanation:

- YEAR(NGAYSINH) extracts the year from the date of birth.
- BETWEEN 1955 AND 1960 selects teachers born from 1955 to 1960 (inclusive).

c. IS NULL and IS NOT NULL

The **IS NULL** and **IS NOT NULL** operators are used to check whether a value is NULL or NOT NULL. Note that comparison operators such as = or <> **cannot** be used to test for NULL values

Example 11: Retrieve teachers who do not have a direct supervisor.

```
SELECT *
```

```
FROM GIAOVIEN
```

```
WHERE MANQL IS NULL
```

Example 12: Retrieve teachers who have a direct supervisor.

```
SELECT *
```

```
FROM GIAOVIEN
```

```
WHERE MANQL IS NOT NULL
```

d. IN and NOT IN

The **IN** and **NOT IN** operators are used to check whether a value **belongs to** or **does not belong to** a specified set of values.

Example 13: Retrieve teachers whose salary is 20,000, 30,000, or 40,000.

```
SELECT *
```

```
FROM GIAOVIEN
WHERE LUONG IN (20000, 30000, 40000)
```

Note: Using **IN** is equivalent to combining multiple conditions with **OR**. For example:

```
SELECT *
FROM GIAOVIEN
WHERE LUONG = 20000 OR LUONG = 30000 OR LUONG = 40000
```

3.1.1 Queries with String Conditions

To work with string data, in addition to the equality operator (=) for exact string matching, SQL provides the **LIKE** operator for **pattern-based (relative) string matching**.

Example 14: Retrieve teachers whose address is in Ho Chi Minh City.

```
SELECT *
FROM GIAOVIEN
WHERE DIACHI LIKE '%TP HCM'
```

Wildcards Used with LIKE

- **%** : represents any sequence of characters
- **_** : represents a single character
- **[]** : represents a range or set of characters

Notes:

- **LIKE 'ab\%cd%'** returns strings that start with **ab%cd**
- **LIKE 'ab\\cd%'** returns strings that start with **ab\cd**

3.1.2 Queries with Date and Time Conditions

SQL provides several built-in functions for working with date and time values:

- **DATEDIFF**: Calculates the difference between two date/time values based on a specified unit (day, month, year, hour, minute, second).
- **DATEPART**: Extracts a specific part of a date/time value (year, month, day, hour, minute, second).

- **YEAR, MONTH, DAY:** Extract the year, month, or day from a date/time value.
- **GETDATE:** Returns the current system date and time

Example 15: Retrieve projects that started after April 30, 2005.

```
SELECT TENDT, CAPQL
FROM DETAI
WHERE datediff(d, TGBD, '4/30/2005') < 0
```

The date is written in the ISO format (YYYY-MM-DD) to avoid ambiguity

Example 16: Retrieve projects that ended at least one week before December 31, 2007.

```
SELECT *
FROM DETAI
WHERE datediff(d, TGKT, '12/31/2007') > 7
```

Example 17: Retrieve projects that started on April 30, 2005

Method 1:

```
SELECT *
FROM DETAI
WHERE TGBD = '4/30/2005'
```

Method 2:

```
SELECT *
FROM DETAI
WHERE datediff(d, TGBD, '4/30/2005') = 0
```

Note: When comparing values of the **DATETIME** data type, using date functions is more accurate. In *Method 1*, if a project has TGBD = '2005-04-30 17:00:00', it will **not** be included in the result set because the time component causes the value to differ from '2005-04-30'.

3.1.3 Using Functions in Queries

SQL allows functions to be used in queries in several ways:

- Using functions in the WHERE clause:
- Functions can be used within conditional expressions to filter data.

- Using functions in the SELECT clause:
- In addition to arithmetic operators such as +, −, *, and /, functions can also be applied to attributes in the SELECT clause.
- Using functions in the ORDER BY clause:
- Functions can be used to sort query results based on computed values
- Common Functions Supported by SQL¹:
 - Date and Time Functions:
 - datediff, datepart
 - year, month, day
 - getdate
 - dateadd
 - String Functions
 - len
 - replace
 - charindex
 - reverse
 - Data Type Conversion Functions
 - convert
 - cast
 - Mathematical Functions
 - floor, ceil
 - ...

Example 18: Retrieve the full name and age of all teachers

```
SELECT HOTEN, datediff(yy, NG SINH, getdate()) as TUOI
FROM GIAOVIEN
```

Or:

```
SELECT HOTEN, year(getdate()) - year(NG SINH) as TUOI
FROM GIAOVIEN
```

Example 19: Retrieve teachers who were born in 1975.

```
SELECT *
FROM GIAOVIEN
```

¹ Students should self-study how to use these functions by consulting Microsoft's documentation

WHERE year(NGSINH) = 1975

Example 20: Retrieve the salary and the salary after a 10% increase for each teacher.

```
SELECT LUONG AS LUONG_TRUOC, LUONG * 1.1 AS LUONG_SAU
FROM GIAOVIEN
```

Example 21: Retrieve the list of project names and their starting year, sorted in descending order by starting year.

```
SELECT TENDT, year(TGBD) AS NAMBD
FROM DETAI
ORDER BY year(TGBD) DESC
```

3.1.4 Supported Set Operators in SQL Server

```
SELECT ... FROM ... WHERE ...
UNION | INTERSECT | EXCEPT
SELECT ... FROM ... WHERE ...
```

Explanation:

- **UNION:** Combines the results of two queries and **removes duplicate rows**.
- **UNION ALL:** Combines the results of two queries and **keeps all rows**, including duplicates.
- **INTERSECT:** Returns only the rows that **appear in both result sets**.
- **EXCEPT:** Returns the rows that appear in the **first query but not in the second**.

Important Notes:

- The number of columns in each **SELECT** statement must be the **same**.
- Corresponding columns must have **compatible data types**.
- The column names in the final result set are taken from the **first SELECT statement**.
- **ORDER BY** can only appear **once**, at the end of the entire set operation.

Example 22: Retrieve information about department heads who participate in projects.

```

SELECT TRBOMON
FROM BOMON
INTERSECT
SELECT MAGV
FROM THAMGIA_DT

```

3.1.5 Join Operations

Join operations are used to retrieve data from **multiple tables** by correctly applying the relationships between tables to form valid join conditions.

Relationships between GIAOVIEN and BOMON:

There are two relationships between the **GIAOVIEN** and **BOMON** tables:

1. **A teacher works in a department**

To determine which department a teacher works for, the relationship between **GIAOVIEN.MABM** and **BOMON.MABM** is used.

2. **A department has a department head**

To identify which teacher is the head of a department, the relationship between **BOMON.TRBOMON** and **GIAOVIEN.MAGV** is used

:

Example 23: Retrieve the teacher name and the name of the department where the teacher works.

```

SELECT HOTEN, TENBM
FROM GIAOVIEN, BOMON
WHERE GIAOVIEN.MABM = BOMON.MABM

```

Example 24: Retrieve the teacher name and the name of the department for which the teacher is the department head.

```

SELECT HOTEN, TENBM
FROM BOMON, GIAOVIEN
WHERE BOMON.TRUONGBM = GIAOVIEN.MAGV

```

3.1.6 Using ALIAS²

When querying data from multiple tables, **aliases** are used to make queries **shorter, clearer, and easier to understand**. Aliases are especially useful in the following cases:

- When different tables contain **columns with the same name**
- When a table is used **more than once** in a query
- When table names are long and need to be shortened

By assigning aliases to tables, we can clearly distinguish between tables and their attributes in a query

Example 25: Retrieve the department name and the name of the head of that department (using ALIAS):

```
SELECT K.TENKHOA, GV.HOTEN
FROM KHOA AS K, GIAOVIEN AS GV
WHERE K.TRKHOA = GV.MAGV
```

Explanation:

- k is an alias for the **KHOA** table.
- gv is an alias for the **GIAOVIEN** table.
- Aliases help shorten table names and make the join condition easier to read

Or:

```
SELECT K.TENKHOA, GV.HOTEN
FROM KHOA K, GIAOVIEN GV
WHERE K.TRKHOA = GV.MAGV
```

Example 26: Retrieve the teacher's name and the names of the teacher's relatives:

```
SELECT GV.HOTEN AS TENG, NT.TEN AS TENNT
FROM GIAOVIEN AS GV, NGUOITHAN AS NT
WHERE NT.MAGV = GV.MAGV
```

² Using ALIAS helps prevent ambiguity between columns with the same name and improves the readability of multi-table queries.

Example 27: Retrieve the teacher's name and the name of the teacher's manager

```
SELECT GV.HOTEN AS TENG, NQL.HOTEN AS TENNQL  
FROM GIAOVIEN AS GV, GIAOVIEN AS NQL  
WHERE GV.MANQL = NQL.MAGV
```

- Example 28: **Retrieve the teacher's name and the name of the faculty to which the teacher belongs.**

```
SELECT GV.HOTEN, K.TENKHOA  
FROM GIAOVIEN GV, BOMON BM, KHOA K  
WHERE GV.MABM = BM.MABM AND BM.MAKHOA = K.MAKHOA
```


3.1.7 Using JOIN

```
SELECT ...  
FROM (TABLE1 JOIN TABLE 2 ON [Điều kiện kết] ) JOIN TABLE3 ON [Điều kiện kết]  
WHERE ...
```

Example 29: (Rewritten Using JOIN) Retrieve the teacher's name and the name of the department where the teacher works.

Using join conditions in the WHERE clause.

```
SELECT G.HOTEN, B.TENBM  
FROM GIAOVIEN G JOIN BOMON B  
WHERE G.MABM = B.MABM
```

Using JOIN

```
SELECT G.HOTEN, B.TENBM  
FROM GIAOVIEN G JOIN BOMON B ON G.MABM = B.MABM
```

4 In-Class Exercises

Requirement: Write SQL queries Q1, Q3, ..., Q25 for the “Quan ly de tai” exercise.

QUẢN LÝ DỰ TÀI

- Q1. Retrieve the full names and salaries of female teachers.
- Q2. Retrieve the full names of teachers and their salaries after a 10% increase.
- Q3. Retrieve the IDs of teachers whose names start with “Nguyễn” and whose salaries are above \$2000, or who are department heads appointed after 1995.
- Q4. Retrieve the names of teachers in the Faculty of Information Technology.
- Q5. Retrieve information about departments along with information about the teachers who serve as department heads.
- Q6. For each teacher, retrieve information about the department in which they work.
- Q7. Retrieve the project names and the teachers who are in charge of those projects.
- Q8. For each faculty, retrieve information about the head of the faculty.
- Q9. Retrieve teachers from the “Vi sinh” department who participate in project 006.
- Q10. For projects managed at the “Thành phố” level, retrieve the project ID, the topic to which

the project belongs, and the full name, date of birth, and address of the project leader.

- Q11. Retrieve the full names of teachers who are directly supervised by “Nguyễn Thanh Tùng”.
- Q12. Retrieve the name of the teacher who is the head of the “Hệ thống thông tin” department.
- Q13. Retrieve the names of project leaders for projects under the “Quản lý giáo dục” topic.
- Q14. Retrieve the names of tasks of the project “HTTT quản lý các trường ĐH” that started in March 2008.
- Q15. Retrieve the teacher’s name and the name of their academic supervisor.
- Q16. Retrieve tasks that started between January 1, 2007 and August 1, 2007.
- Q17. Retrieve the full names of teachers who work in the same department as the teacher “Trần Trà Hương”.
- Q18. Retrieve teachers who are both department heads and project leaders.
- Q19. Retrieve the names of teachers who are both faculty heads and department heads.
- Q20. Retrieve the names of department heads who are also project leaders.
- Q21. **Q22.** Retrieve the IDs of faculty heads who are project leaders.
- Q22. **Q23.** Retrieve the IDs of teachers who belong to the “HTTT” department or who participate in project “001”.
- Q23. Retrieve teachers who work in the same department as teacher 002.
- Q24. Retrieve teachers who are department heads.
- Q25. Retrieve the full names and salaries of teachers

Duration: 1 hour.